Code walkthrough

instruction_register.v

IF/ID Pipeline Register (the IR)

What it does	Holds the instruction being decoded
Where it works	between IF and ID
Size	43 lines, explained in 3 small parts
Modules in this file	instruction_register
Calls (uses)	nothing (no other modules)
Called by	cpu_top.v - instance IR (the IF/ID register)

This is the first pipeline register. Every clock it stores the instruction just fetched, so the ID stage can work on it while the IF stage fetches the next one. It also stores the instruction's address and a valid bit.

Map of the file

Part	Lines	What this part is
1	1-19	Header comment
2	20-30	Ports
3	31-43	Clocked behaviour

> How to read this document: each part below shows a few lines of the real file, with their line numbers, followed by a plain-language explanation. Read the parts in order to understand the whole file without scrolling through it in one go.

Part 1 · Header comment (lines 1-19)

instruction_register.v lines 1-19

```
 1  // ======================================================================
 2  // FILE: instruction_register.v
 3  // The IR is the IF/ID PIPELINE REGISTER: it sits between the
 4  // FETCH stage and the DECODE stage.
 5  //
 6  // Interface (as in the project plan):
 7  //   clk, rst, ir_load, instruction_in, instruction_out
 8  // Extra pipeline signals:
 9  //   flush           : 1 = throw the instruction away (taken jump)
10  //   pc_in / pc_out  : address of the instruction (to follow it in the
11  //                     pipeline and in the waveform)
12  //   valid_out       : 1 = holds a real instruction, 0 = empty (bubble)
13  //
14  // Every clock (ir_load = 1) it saves the instruction just fetched.
15  //
16  // CONNECTIONS
17  //   Calls       : nothing (no other modules)
18  //   Called by   : cpu_top.v - instance IR (the IF/ID register)
19  // ======================================================================
```

- Explains the role (IF/ID register), the plan's interface, and the extra pipeline signals: flush, pc_in/pc_out and valid_out.
- The **CONNECTIONS** lines at the end of the header list which other files this file uses (Calls) and which files use it (Called by).

Part 2 · Ports (lines 20-30)

instruction_register.v lines 20-30

```
20  module instruction_register (
21      input             clk,
22      input             rst,
23      input             ir_load,          // 1 = save the new instruction
24      input             flush,            // 1 = replace it with a NOP
25      input      [11:0] instruction_in,   // from instruction memory
26      input      [3:0]  pc_in,            // its address
27      output reg [11:0] instruction_out,  // to the DECODE stage
28      output reg [3:0]  pc_out,
29      output reg        valid_out
30  );
```

- `ir_load`: load a new instruction (tied to 1 in cpu_top).
- `flush`: when 1, throw away the stored instruction (a jump was taken).
- `pc_in/pc_out`: the address of the instruction, carried along for tracing.
- `valid_out`: 1 = real instruction, 0 = empty slot (a "bubble").

Part 3 · Clocked behaviour (lines 31-43)

instruction_register.v lines 31-43

```
31      always @(posedge clk) begin
32          if (rst || flush) begin
33              instruction_out <= 12'hC00;     // C00 = reserved opcode = NOP
34              pc_out          <= 4'd0;
35              valid_out       <= 1'b0;        // empty slot (bubble)
36          end
37          else if (ir_load) begin
38              instruction_out <= instruction_in;
39              pc_out          <= pc_in;
40              valid_out       <= 1'b1;
41          end
42      end
43  endmodule
```

- On reset **or** flush: store C00 (NOP), address 0 and valid = 0. The next stage then sees "nothing to do".
- Otherwise, if ir_load is 1: store the fetched instruction, its address, and valid = 1.

In short: Every clock: fetched instruction → IR; on a taken jump: IR → empty. **Example:** the memory gives 683 from address 1. At the clock edge instruction_out = 683, pc_out = 1, valid_out = 1. If flush = 1 at that edge instead, instruction_out = C00 and valid_out = 0.